

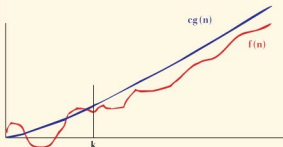
NOTATIONS

All the definitions given here are provided by the National Institute of Standards and Technology.

O Notation

This is a theoretical measure of the execution of an algorithm, usually the time or memory needed, given the problem size n , which is usually the number of items. Informally, saying some equation $f(n) = O(g(n))$ means it is less than some constant multiple of $g(n)$. The notation is read, "f of n is big oh of g of n".

Formal definition: $f(n) = O(g(n))$ means there are positive constants c and k , such that $0 \leq f(n) \leq cg(n)$ for all $n \geq k$. The values of c and k must be fixed for the function f and must not depend on n .



Also known as O .

Ω notation

This is a theoretical measure of the execution of an algorithm, usually the time or memory needed, given the problem size n , which is usually the number of items. Informally, saying some equation $f(n) = \Omega(g(n))$ means it is more than some constant multiple of $g(n)$.

Formal definition: $f(n) = \Omega(g(n))$ means there are positive constants c and k , such that $0 \leq cg(n) \leq f(n)$ for all $n \geq k$. The values of c and k must be fixed for the function f and must not depend on n .

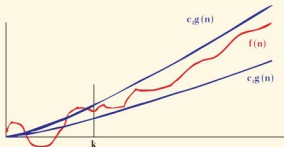
ω notation

This is a theoretical measure of the execution of an algorithm, usually the time or memory needed, given the problem size n , which is usually the number of items. Informally, saying some equation $f(n) = \omega(g(n))$ means $g(n)$ becomes insignificant relative to $f(n)$ as n goes to infinity.

Formal definition: $f(n) = \omega(g(n))$ means that for any positive constant c , there exists a constant k , such that $0 \leq cg(n) < f(n)$ for all $n \geq k$. The value of k must not depend on n , but may depend on c .

Θ Notation

This is a theoretical measure of the execution of an algorithm, usually the time or memory needed, given the problem size n , which is usually the number of items. Informally, saying some equation $f(n) = \Theta(g(n))$ means it is within a constant multiple of $g(n)$. The equation is read as, "f of n is theta g of n".



Formal definition: $f(n) = \Theta(g(n))$ means there are positive constants c_1 , c_2 , and k , such that $0 \leq c_1g(n) \leq f(n) \leq c_2g(n)$ for all $n \geq k$. The values of c_1 , c_2 , and k must be fixed for the function f and must not depend on n .

o notation

This is a theoretical measure of the execution of an algorithm, usually the time or memory needed, given the problem size n , which is usually the number of items. Informally, saying some equation $f(n) = o(g(n))$ means $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity. The notation is read as, "f of n is little oh of g of n".

Formal definition: $f(n) = o(g(n))$ means for all $c > 0$ there exists some $k > 0$ such that $0 \leq f(n) < cg(n)$ for all $n \geq k$. The value of k must not depend on n , but may depend on c .

Resources:

- Finding the Big-O of a function: www.eecs.harvard.edu/~ellard/Q-97/HTML/root/node7.html
- Big-O notation: en.wikipedia.org/wiki/Big_O_notation

```
fi

if [ check_month_name $word2 ]; then
echo "Invalid date format: unknown month name specified.">&2
exit 1
fi

word2="$(normalize $word2)"

if [ -z "$word3" ]; then
```

```
date="$word1$word2"
else
if [ ! -z "$(echo $word3 | sed 's/[[[:digit:]]//g')"]; then
echo "Invalid date format: third field should be a year.">&2
exit 1
elif [ $word3 -lt 2009 -o $word3 -gt 3000 ]; then
echo "Invalid date format: year value should be between 2009-3000"
>&2
exit 1
fi
```